Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Elektro- und Informationstechnik

Elektro- und Informationstechnik (Bachelor)

Laborbericht Regelungstechnik

Versuch 01 - Einführung in MatLab

Rishabh Venugopal

Gruppennummer:	1
Gruppenmitglieder:	Rishabh Venugopal (96535), Ruth Riebel (86074)
Betreuer:	Prof. Dr.-Ing. Dirk Feßler und Bernd Walenta
Datum:	9. April 2026

Inhaltsverzeichnis

1 Aufgabe 1 ... 3

1.1 Plotten von zwei Sinussignalen ... 3

1.2 Plotten von zwei Sinussignalen ... 4

1.3 Lissajous-Figur ... 6

1.4 Lissajous-Figur mit geringerer Sampling Rate ... 7

2 Aufgabe 2 ... 8

2.1 Vorbereitung ... 8

2.2 Grenzfrequenz in MatLab beschreiben ... 8

2.3 Bode-Diagramm ... 9

2.4 Nyquist-Diagramm ... 10

3 Aufgabe 3 ... 13

3.1 Aufgabenstellung ... 13

3.2 Modellierung in SimuLink ... 14

3.2.1 Einstellung der Führungsgröße ... 14

3.2.2 Modellierung des Reglers ... 16

3.2.3 Modellierung der Heizung ... 16

3.2.4 Rückkopplung ... 17

3.3 Simulationsergebnisse ... 17

3.4 Variation von Ein- und Ausschalt-Threshold ... 17

1 Aufgabe 1

1.1 Plotten von zwei Sinussignalen

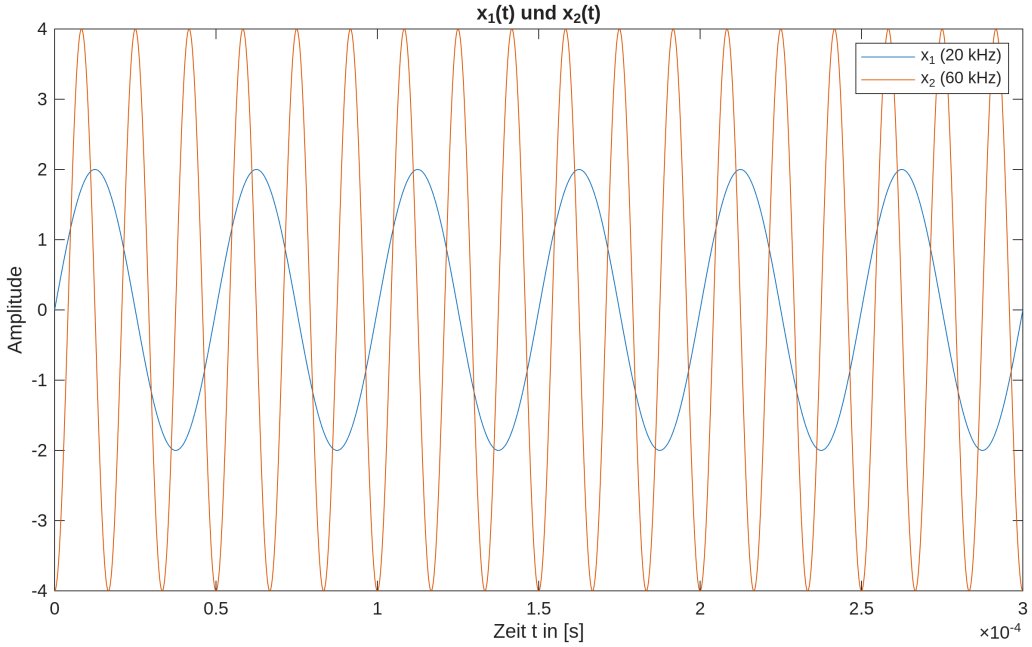

Abbildung 1: x1 und x2

Es galt, zwei Sinussignale $x_1(t)$ und $x_2(t)$ in einem Diagramm grafisch darzustellen.

$$x_1(t) = x_{1,0} \cdot \sin(2\pi f_1 t) \tag{1}$$

$$x_2(t) = x_{2,0} \cdot \sin\left(2\pi f_2 t - \frac{\pi}{2}\right) \tag{2}$$

Mit $f_1 = 20kHz$, $f_2 = 60kHz$, $x_{1,0} = 2$ und $x_{2,0} = 4$.
Damit die Signale $x_1(t)$ und $x_2(t)$ richtig dargestellt werden, muss die Abtastfrequenz mindestens doppelt so hoch sein wie die höchste Frequenz der Signale sein (Nyquist-Shannon-Abtasttheorem). Hier wurde eine Abtastfrequenz von $f = 10MHz$ gewählt, welches deutlich höher ist als $2 \cdot 60kHz$ ist.

Quellcode 1.1.1: Matlab Code zur Parameterwahl und Zeitachse

```
x1_0 = 2;
x2_0 = 4;
f1 = 20e3;
f2 = 60e3;

t = 0 : 0.1e-6 : 0.3e-3;     % Abtastfrequenz f=10MHz
```

Quellcode 1.1.2: Matlab Code zur Berechnung von $x_1(t)$ und $x_2(t)$

```
x1 = x1_0 * sin(2*pi*f1*t);
x2 = x2_0 * sin(2*pi*f2*t - pi/2);
```

Quellcode 1.1.3: Matlab Code um die Signale zu plotten

```
figure;
plot(t, x1)
hold on;
plot(t, x2)
hold off;

title('x_1(t) und x_2(t)');
xlabel('Zeit t in [s]');
ylabel('Amplitude');
legend('x_1 (20 kHz)', 'x_2 (60 kHz)');
```

1.2 Plotten von zwei Sinussignalen

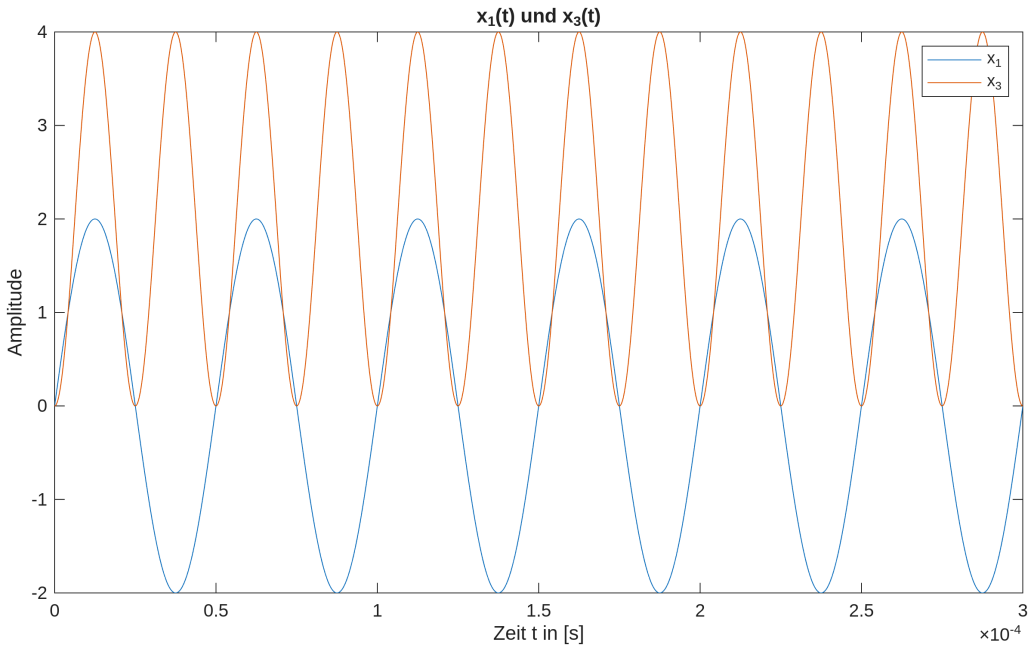

Abbildung 2: Summe der Signale x1 und x3

Hier zu sehen ist $x_1(t)$ und $x_3(t)$.

$$\begin{aligned} x_3(t) &= x_1(t) \cdot x_1(t) \\ &= x_{1,0}^2 \cdot \sin^2(2\pi f_1 t) \end{aligned} \tag{3}$$

Quellcode 1.2.1: Matlab Code zur Berechnung von $x_3(t)$

```
x3 = x1 .* x1;
```

Quellcode 1.2.2: Matlab Code zum Vergleich von $x_1(t)$ und $x_3(t)$

```
plot(t, x1)
hold on;
plot(t, x3)
hold off;
title('x_1(t) und x_3(t)');
xlabel('Zeit t in [s]');
ylabel('Amplitude');
legend('x_1', 'x_3');
```

Da $sin^2(\phi) = \frac{1-\cos(2\phi)}{2}$, kann man $x_3(t)$ auch noch weiter umschreiben:

$$\begin{aligned} x_3(t) &= \frac{x_{1,0}^2}{2} \cdot (1 - \cos(4\pi f_1 t)) \\ &= \frac{x_{1,0}^2}{2} - \frac{x_{1,0}^2}{2} \cdot \cos(4\pi f_1 t) \end{aligned} \tag{4}$$

$$\Rightarrow x_3(t) = \underbrace{\frac{x_{1,0}^2}{2}}_{\text{Gleichanteil}} - \underbrace{\frac{x_{1,0}^2}{2} \cdot \cos(2\pi \cdot (2f_1) \cdot t)}_{\text{doppelte Frequenz}}$$

Somit ist $x_3(t)$ die Summe eines Gleichanteils und eines Signals mit der doppelten Frequenz von $x_1(t)$.

In Matlab kann $x_3(t)$ wie folgt angezeigt werden:

Quellcode 1.3.1: Matlab Code zur Darstellung der Lissajous-Figur

```
plot(x1,x2)
title('Lissajous-Figur');
xlabel('x_1(t)');
ylabel('x_2(t)');
```

1.3 Lissajous-Figur

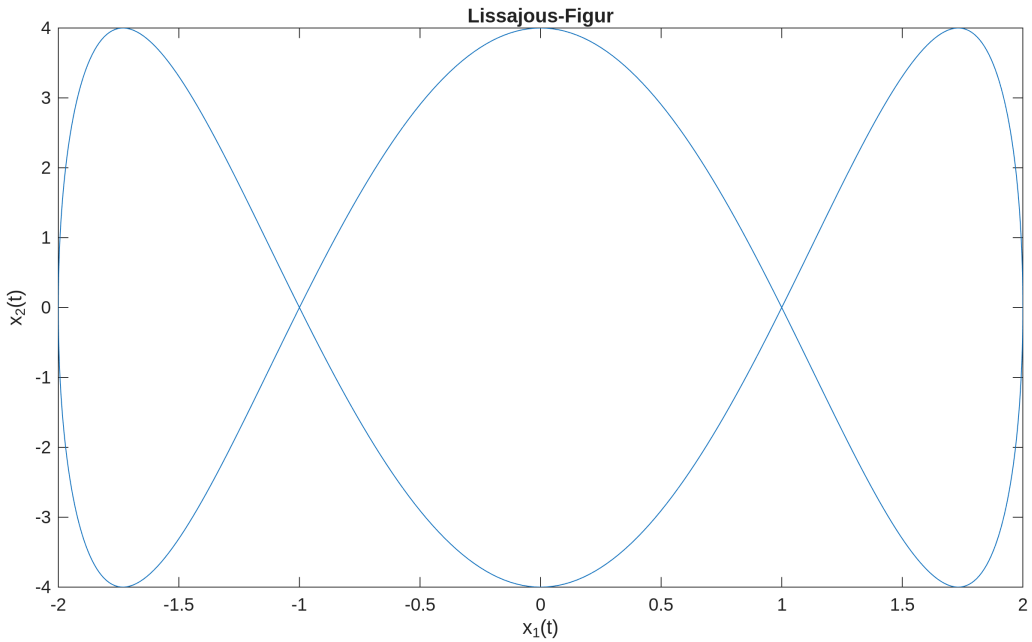

Abbildung 3: Lissajous-Figur von x_1 und x_2

Hier zu sehen ist die Lissajous-Figur von $x_1(t)$ und $x_2(t)$ Um eine Lissajous-Figur zu erstellen, müssen die beiden Signale $x_1(t)$ und $x_2(t)$ um $\varphi = 90^\circ$ phasenverschoben sein, damit die Figur eine geschlossene Form annimmt. Hierbei wird $x_1(t)$ über $x_2(t)$ geplottet, also $x_1(t)$ auf der x-Achse und $x_2(t)$ auf der y-Achse. Mit der Lissajous-Figur kann man Phasenverschiebungen zwischen den Signalen erkennen. Früher wurde diese Methode verwendet, um bei Elektronenröhrenoszilloskopen die Phasenverschiebung zwischen zwei Signalen zu bestimmen. Heutzutage können digitale Oszilloskope die Phasenverschiebung direkt anzeigen, sodass die Lissajous-Figur eher eine anschauliche Darstellung ist. Zudem kann man die Frequenzverhältnisse der Signale erkennen. In diesem Fall ist das Frequenzverhältnis 1:3 (ablesen an der Anzahl der Nullstellen der Signale). In Matlab wurde die Lissajous-Figur wie folgt dargestellt:

Quellcode 1.4.1: Matlab Code zur Signalerzeugung bei zu niedriger Abtastfrequenz

```
t=linspace(0,2,1000);
f1=20e3;
f2=60e3;

x1 = x1_0 * sin(2*pi*f1*t);
x2 = x2_0 * sin(2*pi*f2*t - pi/2);
```

1.4 Lissajous-Figur mit geringerer Sampling Rate

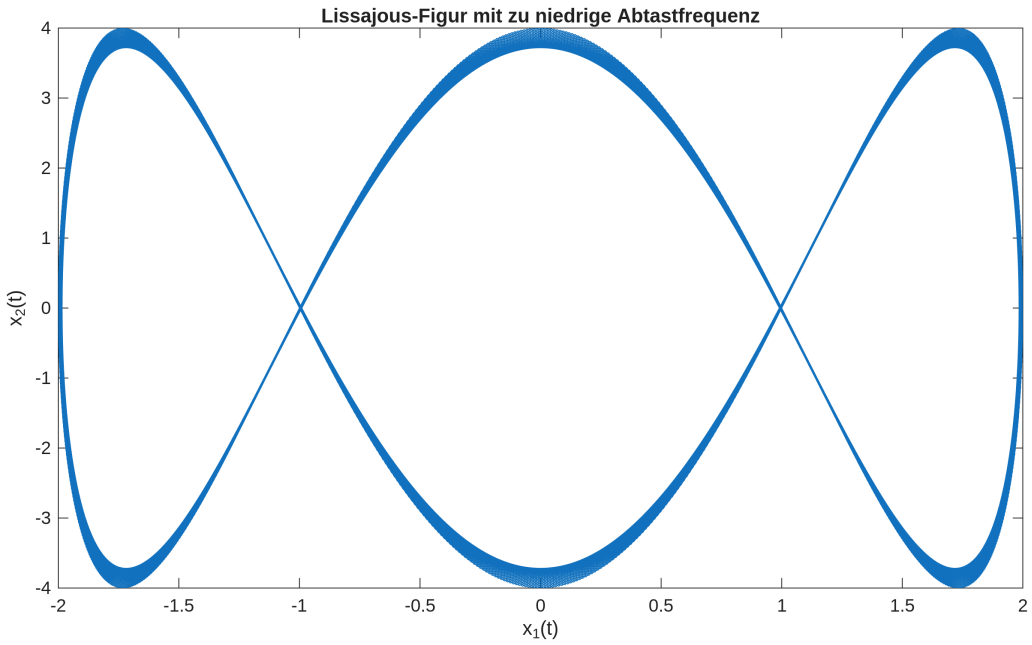

Abbildung 4: Lissajous-Figur von x_1 und x_2 mit geringerer Sampling Rate

Hier zu sehen ist die Lissajous-Figur von $x_1(t)$ und $x_2(t)$, jedoch mit einer geringeren Abtastfrequenz von $f = 500Hz$. Da die Abtastfrequenz zu gering ist, kommt es zu Aliasing und das Signal kann nicht mehr korrekt dargestellt werden. Auf einem Plot von $x_1(t)$ über $x_2(t)$ würden zu niedrige Frequenzen bzw. eine nicht-konstante Frequenz erscheinen. Das führt zu einer verzerrten Lissajous-Figur wie in Abbildung 4.

Quellcode 1.4.2: Matlab Code zur Darstellung der verzerrten Lissajous-Figur

```
plot(x1, x2);
title('Lissajous-Figur mit zu niedrige Abtastfrequenz')
xlabel('x_1(t)');
ylabel('x_2(t)');
```

2 Aufgabe 2

2.1 Vorbereitung

Die Übertragungsfunktion eines RC-Tiefpassfilters lautet:

$$G(s) = \frac{1}{1 + sRC} \tag{5}$$

Da die Grenzfunktion asympotisch stabil ist, können wir es in den Frequenzbereich überführen:

$$G(s) = \frac{1}{1 + sRC}\bigg|_{s=j\omega} = \frac{1}{1 + j\omega RC} \tag{6}$$

Die Grenzfrequenz f_g eines RC-Tiefpassfilters ist definiert als die Frequenz, bei der die Amplitude des Ausgangssignals um 3dB abgefallen ist. Sie kann mit der folgenden Formel berechnet werden:

$$f_g = \frac{1}{2\pi RC} \tag{7}$$

Da wir R und C suchen, stellen wir um. Hier wird nach C umgestellt und R auf einen festen Wert von $R = 10k\Omega$ gesetzt:

$$C = \frac{1}{2\pi R f_g} \tag{8}$$

$$= \frac{1}{2\pi \cdot 10k\Omega \cdot 10kHz} \approx 1.59nF \tag{9}$$

2.2 Grenzfrequenz in MatLab beschreiben

Quellcode 2.2.1: Matlab Code zur Beschreibung der Übertragungsfunktion

```
f_grenz = 10e3;      % 10kHz
R = 10e3;    % 10kOhm
C = 1 / (2 * pi * f_grenz * R);

num = [1];
denum = [R*C, 1];

G = tf(num, denum)
```

Die Übertragungsfunktion kann in MatLab mit der Funktion `tf` beschrieben werden.

Hierbei ist auf die spezielle Syntax der Funktion `tf` zu achten. Die `tf` Funktion erwartet zwei Arrays mit jeweils die Koeffizienten des Zählers und des Nenners der Übertragungsfunktion. Die Koeffizienten müssen in absteigender Potenz von s angegeben werden.

Betrachten wir als Beispiel die Übertragungsfunktion $G(s) = \frac{67s+420}{69s^2+42s+1}$. Der Zähler muss als `[67, 420]` und der Nenner als `[69, 42, 1]` angegeben werden.
Machen wir das für unseren RC-Tiefpass, so erhalten wir die folgenden Übertragungsfunktion:

Konsolenausgabe 2.2.1: Übertragungsfunktion des RC-Tiefpasses

```
  G =

        1
---------------
1.592e-05 s + 1
```

2.3 Bode-Diagramm

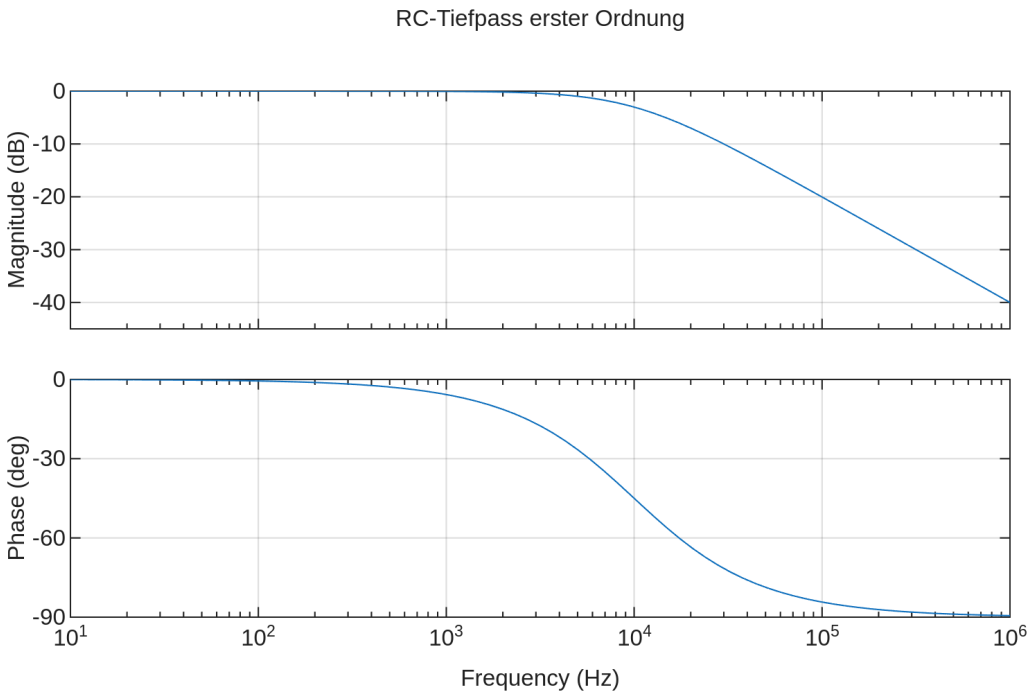

Abbildung 5: Bode-Diagramm des RC-Tiefpassfilters

Abbildung 5 zeigt das Bode-Diagramm des RC-Tiefpassfilters. Es zeigt die Amplitude und die Phase der Übertragungsfunktion über einem Frequenzintervall von $10Hz \leq f \leq 1MHz$. Damit wir ein Intervall von genau $10Hz \leq f \leq 1MHz$ abbilden, müssen wir noch die Grenzen definieren mit `grenzen = { <untere Grenze>, <obere Grenze> }`. Da die Frequenz in standardmäßig in Rad/s angegeben wird, müssen wir die Option `FreqUnits` auf `Hz` setzen.

Quellcode 2.3.1: Bode-Diagramm plotten

```
grenzen = {2*pi*10, 2*pi*1e6}; % Geschweifte Klammer für Intervall
bodeOpts = bodeoptions;
bodeOpts.FreqUnits = 'Hz';
bodeOpts.Grid = 'on';

bodeplot(G, grenzen, bodeOpts)
title("RC-Tiefpass erster Ordnung")
xlabel("Frequenz f")
```

2.4 Nyquist-Diagramm

Ein Nyquist-Diagramm ist, wie auch das Bode-Diagramm, eine grafische Darstellung der Übertragungsfunktion eines LTI-Systems.
Ein Bodediagramm zeigt die Amplitude und Phase und trägt diese jeweils über der Frequenz auf:

$$\begin{aligned} \text{Amplitude} &= |G(j\omega)| \\ \text{Phase} &= \angle G(j\omega)) \end{aligned}$$

Ein Nyquist-Diagramm zeigt die Real- und Imaginärteile der Übertragungsfunktion:

$$\begin{aligned} \text{Realteil} &= \Re G(j\omega) \\ \text{Imaginärteil} &= \Im G(j\omega) \end{aligned}$$

Der Imaginärteil wird über den Realteil aufgetragen, also $\Im G(j\omega)$ auf der y-Achse und $\Re G(j\omega)$ auf der x-Achse.

In unserem Fall:

$$\begin{aligned} \Re G(j\omega) &= \frac{1}{1+(\omega RC)^2} \\ \Im G(j\omega) &= \frac{-\omega RC}{1+(\omega RC)^2} \end{aligned}$$

Man zeichnet die Kurve für ω von 0 bis ∞, gekennzeichnet durch die Pfeile in der Kurve.

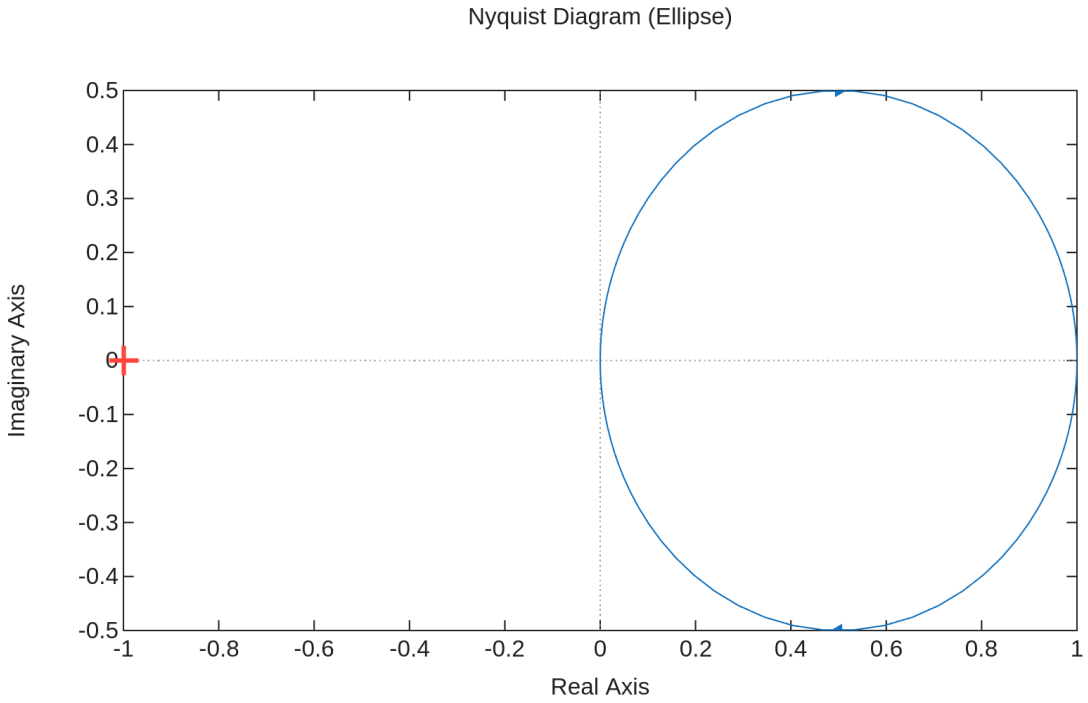

Abbildung 6: Nyquist-Diagramm des RC-Tiefpassfilters

Quellcode 2.4.1: Nyquist-Diagramm Ellipse plotten

```
figure;
npOpts = nyquistoptions;
nyquistplot(G, npOpts)
title("Nyquist Diagram (Ellipse)")
```

Man erkennt in Abbildung 6 jedoch, dass der Nyquist-Plot eine Ellipse ist, anstatt wie üblich ein Kreis. Dies liegt daran, dass die Achsen nicht im gleichen Maßstab skaliert sind. Um dies zu beheben, müssen wir nur die Option `axis equal` hinzufügen.

Quellcode 2.4.2: Nyquist-Diagramm als Kreis plotten

```
figure;
npOpts = nyquistoptions;
nyquistplot(G, npOpts)
axis equal;     % <-- Achsen im gleichen Maßstab
title("Nyquist Diagram (Kreis)")
```

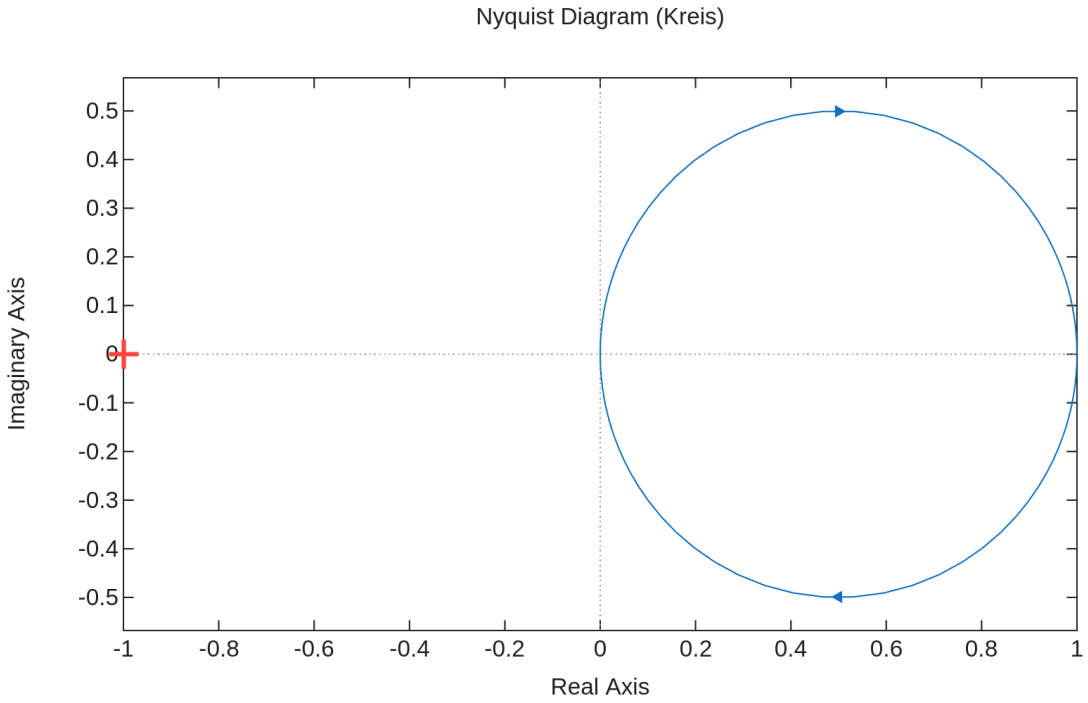

Abbildung 7: Nyquist-Diagramm des RC-Tiefpassfilters (Kreis)

Uns ist jedoch bekannt, dass die Übertragungsfunktion eines RC-Tiefpassfilters ein halbkreis im vierten Quadranten ist, jedoch wird in Abbildung 7 ein voller Kreis dargestellt. Dies liegt daran, dass die Nyquist-Plot Funktion von MatLab die Frequenz von $-\infty$ bis ∞ abbildet, anstatt von 0 bis ∞. Um dies zu beheben, müssen wir die Option `npOpts.ShowFullContour = 'off';` hinzufügen.

Quellcode 2.4.3: Nyquist-Diagramm als Halbkreis plotten

```
f = figure;
npOpts.ShowFullContour = 'off';      % <-- exclude negative frequencies
nyquistplot(G, npOpts)
axis equal;      % <-- Damit es als Hlabkreis dargestellt wird und nicht als halbellipse
title("Nyquist Diagram (Halbkreis)")
```

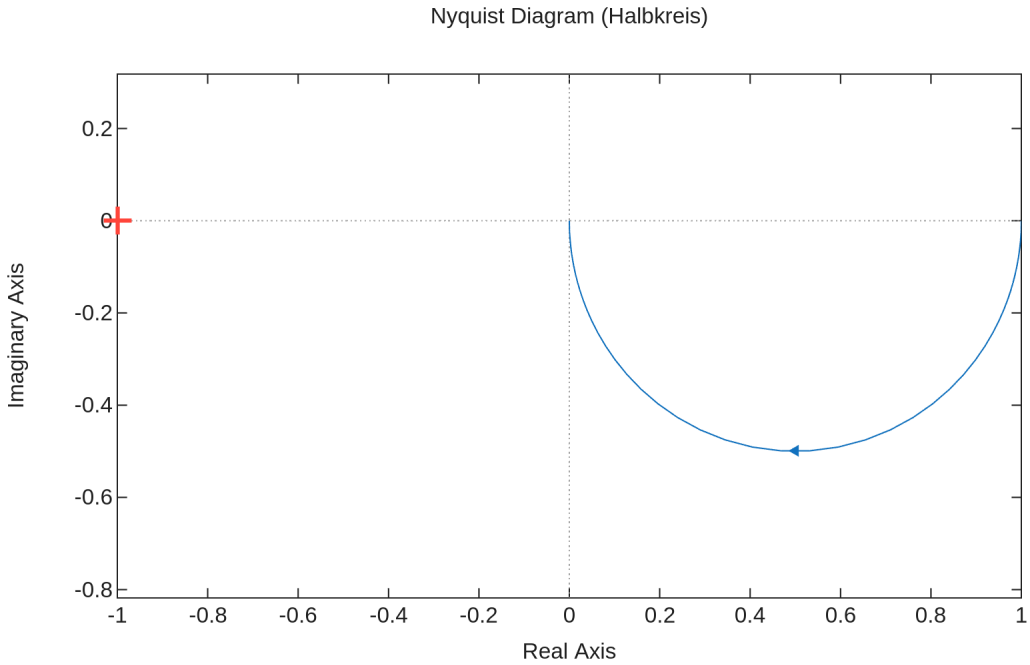

Abbildung 8: Nyquist-Diagramm des RC-Tiefpassfilters (Halbkreis)

3 Aufgabe 3

3.1 Aufgabenstellung

In dieser Aufgabe soll eine Ofenregelung mit Matlab/Simulink modelliert und simuliert werden. Die Regelstrecke sieht wie folgt aus:

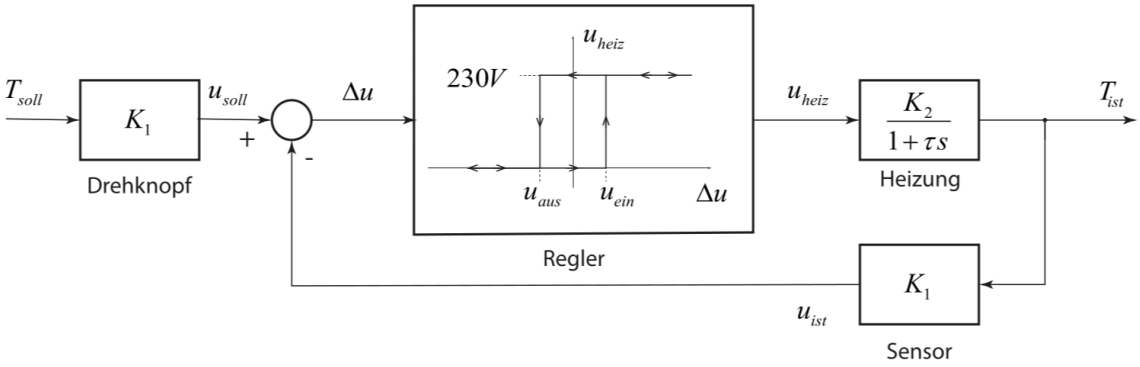

Abbildung 9: Zu bauende Ofenregelstrecke

Hierfür wurden in einer .m Datei die folgenden Parameter für unsere Strecke definiert:

Quellcode 3.1.1: Parameter für die Übertragungsfunktion

```
K1 = 10 / 400;       % 10V/400°C
K2 = 400 / 230;      % 400°C/230V
tau = 10 * 60;
u_ein = 0.75;
u_aus = -0.75;
```

3.2 Modellierung in SimuLink

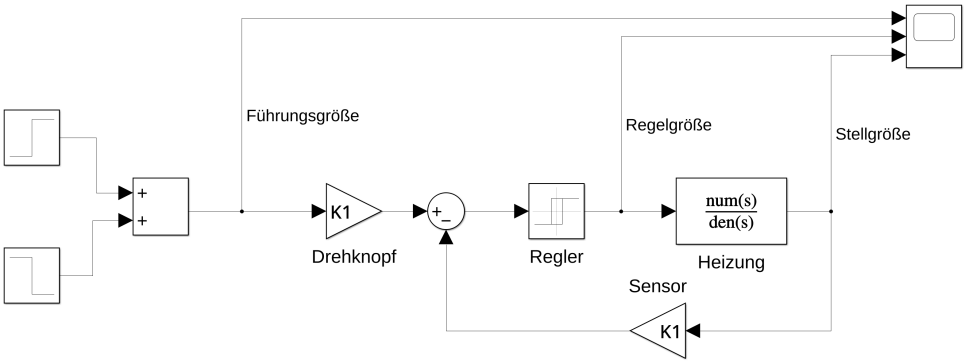

Abbildung 10: Simulink-Modell der Ofenregelstrecke

3.2.1 Einstellung der Führungsgröße

Der Ofen soll laut Aufgabenstellung auf 180°C für 30min aufgeheizt werden. Danach soll die Führungsgröße auf wieder auf 0°C gesetzt werden. Hierfür wurde in Simulink zwei Step-Blöcke verwendet (siehe Abbildung 11).

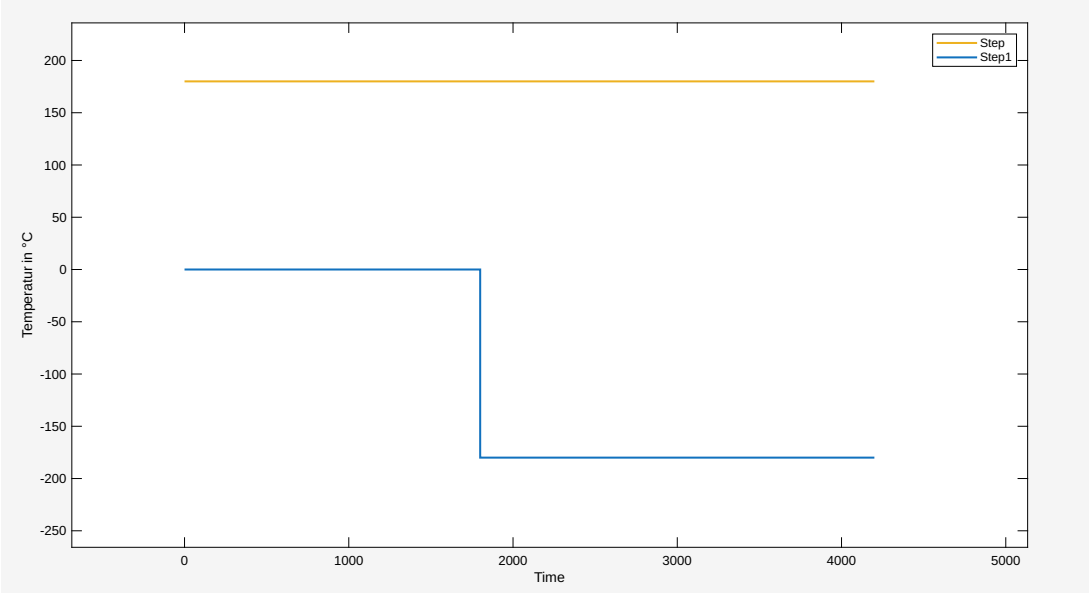

Abbildung 11: Einstellung der Führungsgröße mit Step-Blöcken

Durch die Addition der beiden Blöcke entsteht unsere gewünschte Führungsgröße mit einem Sprung von 0°C auf 180°C bei t=0s und einem Sprung von 180°C auf 0°C bei t=1800s (siehe Abbildung 12). Um die Führungsgröße in eine für den Regler brauchbare Spannung umzuwandeln, wurde ein Gain-Block mit einem Verstärkungsfaktor von K_1 verwendet.

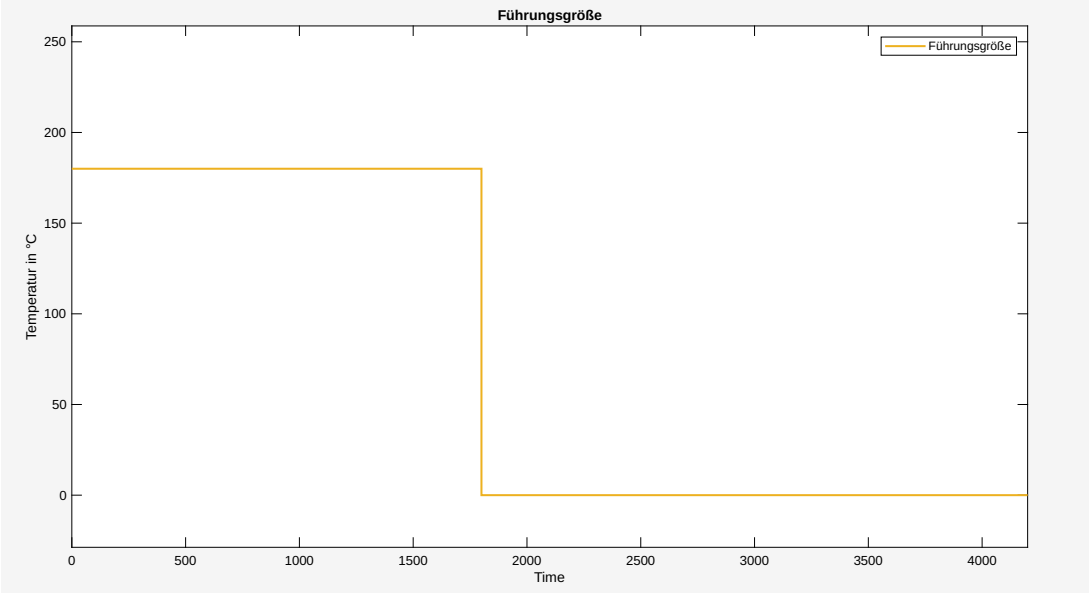

Abbildung 12: Führungsgröße über die Zeit

3.2.2 Modellierung des Reglers

Der Regler selbst ist ein einfacher Relais, welche 230V oder 0V ausgibt, je nach Eingangssignal. Das Eingangssignal wird aus der Differenz aus Führungsgröße und Istwert (über den Sensor gemessen) gebildet. Der Relais-Block in Simulink hat die folgenden Einstellungen:

- **`Switch on point:`** `u_ein`
- **`Switch off point:`** `u_aus`
- **`Output when on:`** 230
- **`Output when off:`** 0

3.2.3 Modellierung der Heizung

Die Heizung ist eine Übertragungsfunktion

$$G(s) = \frac{K_2}{1 + \tau s} \tag{10}$$

Diese Übertragungsfunktion wurde mit dem gleichen Syntax wie in Quellcode 2.2.1 beschrieben.

3.2.4 Rückkopplung

Die Rückkopplung der Strecke erfolgt über einen Sensor, welcher die aktuelle Temperatur misst und in eine Spannung umwandelt. Hierfür wurde ein Gain-Block mit einem Verstärkungsfaktor von K_1 verwendet.

3.3 Simulationsergebnisse

Die Simulationsergebnisse für die Ofenregelstrecke sind in Abbildung 13 dargestellt. Die gelbe Kurve zeigt die Führungsgröße, die orange Kurve zeigt die aktuelle Temperatur des Ofens und die blaue Kurve zeigt die Spannung, welche der Regler ausgibt.

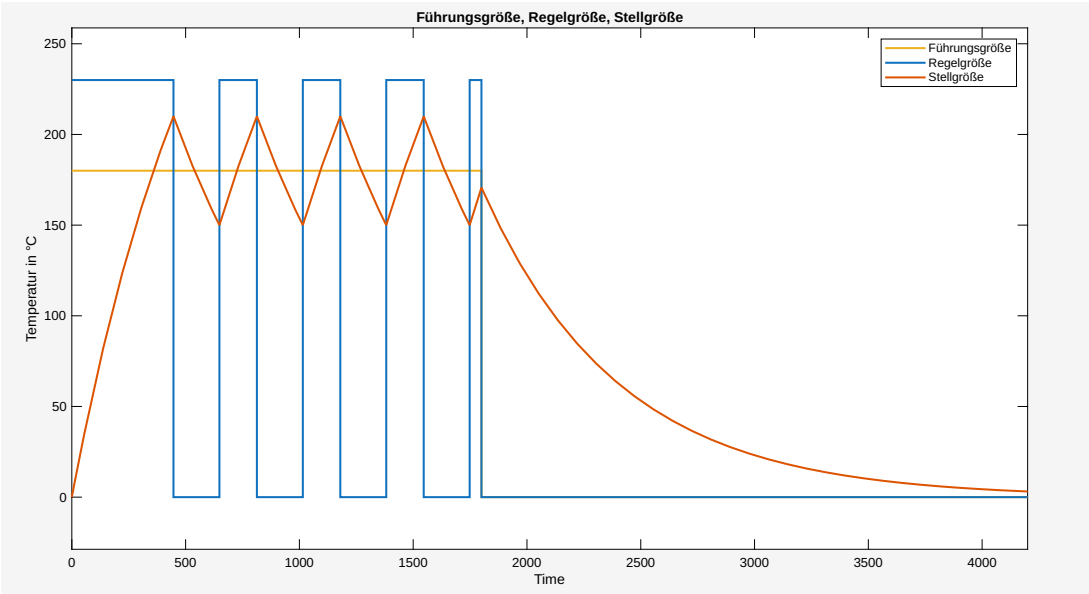

Abbildung 13: Simulationsergebnisse der Ofenregelstrecke

Man kann hier klar erkennen, dass der Regler nicht in der Lage ist, die Führungsgröße von 180°C zu halten. Die Temperatur steigt zwar an, aber da der Regler nur 230V oder 0V ausgeben kann, d.h. nur ein Ein/Ausschalten kann, kann die Temperatur nicht stabil gehalten werden und schwankt. Da wir die An- und Ausschaltbedingungen des Relais auf $\pm 0.75V$ gesetzt haben, schaltet der Regler bei 210°C ein und bei 150°C aus.

3.4 Variation von Ein- und Ausschalt-Threshold

Die Schaltpunkte des Relais können durch die Parameter u_{ein} und u_{aus} angepasst werden. Wenn wir u_{ein}/u_{aus} niedriger machen, schaltet der Regler früher ein oder aus,

was zu einer höheren Frequenz des Ein/Aus-Schaltens führt, also zu mehr kleineren Schwingungen). Dies kann man klar in der folgenden Abbildung erkennen, in welcher u_{ein} auf 0.25V und u_{aus} auf -0.25V gesetzt wurden. Der Regler sollte hier bei 190°C einschalten und bei 170°C ausschalten.

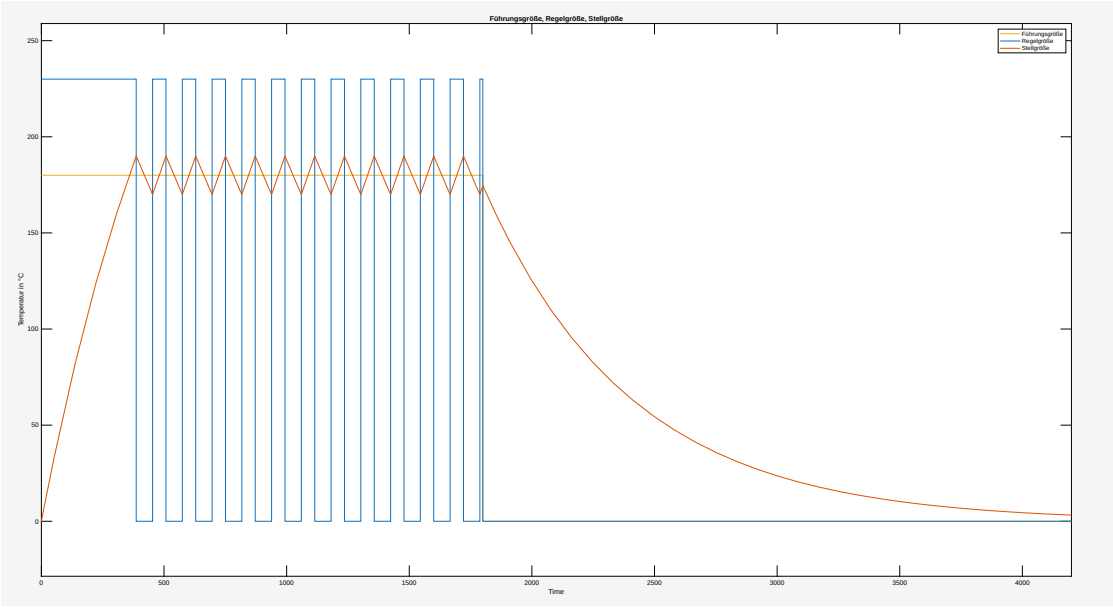

Abbildung 14: Simulationsergebnisse mit angepassten Schaltpunkten

Dieser Regler ist natürlich nicht optimal, da so eine hohe Frequenz den Relais beschädigen würde. Jedoch kann man es leicht als Einführung in SimuLink nehmen.